



Programming Fundamentals

Introduction to Iteration

Programming Fundamentals Team

SETU

2026-09-28





0.1 Where Are We Now?

0.1 Where Are We Now?

Over the last few weeks, we have been building our Python toolkit.

We can now:

- store information using **variables**
- work with different **data types**
- make decisions using **selection**
- organise our code using **functions**

We can already write programs that take input, make decisions and produce useful results.



0.2 Variables and Data Types

0.2 Variables and Data Types

Variables allow us to store information while our program is running.

```
name = "Charlie"  
age = 4  
weight = 28.5  
vaccinated = True
```

We have used different data types:

- str
- int
- float
- bool

And variables can change:



0.2 Variables and Data Types

```
age = age + 1
```



0.3 Selection – Making Decisions

0.3 Selection – Making Decisions

We learned how our programs can make decisions.

```
if age < 1:  
    print("Puppy")  
elif age < 8:  
    print("Adult dog")  
else:  
    print("Senior dog")
```

Selection asks a question:

Is this condition True or False?

The answer determines which code runs.



0.4 Functions – Organising Our Code

0.4 Functions – Organising Our Code

Functions allow us to give a piece of code a job.

```
def calculate_cost(days, daily_price):  
    cost = days * daily_price  
    return cost
```

```
total = calculate_cost(3, 25.0)
```

Functions help us organise and reuse code, and break a larger problem into smaller problems.



0.5 We Can Already Do Quite a Lot

0.5 We Can Already Do Quite a Lot

```
name = input("Dog's name: ")
age = int(input("Dog's age: "))

if age < 1:
    print(name, "is a puppy")
else:
    print(name, "is not a puppy")
```

This program:

- gets input
- stores values
- makes a decision
- displays a result



0.5 We Can Already Do Quite a Lot

But what happens if we want to do it again?



0.6 A New Problem

0.6 A New Problem

Suppose we want our program to print `Welcome!` five times.

```
print("Welcome!")  
print("Welcome!")  
print("Welcome!")  
print("Welcome!")  
print("Welcome!")
```

This works — but what if we wanted it **100 times**?

What if we didn't know in advance how many times we needed to repeat something?



0.7 Another Problem

0.7 Another Problem

```
password = input("Enter password: ")
```

```
if password == "python":  
    print("Access granted")  
else:  
    print("Incorrect password")
```

If the password is wrong, the program ends.

What if we want to keep asking until the user enters the correct password?

We need a way of **repeating code**.



0.8 Iteration

0.8 Iteration

Iteration means repeating a set of instructions.

We often call this a **loop**.

Instead of writing the same code again and again, we tell Python to repeat it for us.

Iteration is one of the fundamental building blocks of programming.



0.9 Three Fundamental Ideas

0.9 Three Fundamental Ideas

Sequence → Selection → Iteration

Sequence — do instructions in order.

Selection — decide which instructions to perform.

Iteration — repeat instructions.

These ideas appear again and again in the programs we write.



0.10 Iteration in Everyday Life

0.10 Iteration in Everyday Life

We use repetition all the time.

Brushing your teeth

Repeat brushing movements until enough time has passed.

Washing dishes

While there are dirty dishes, wash another dish.

Taking attendance

For each student, record whether they are present.

Programming works in much the same way.



0.11 Two Types of Loop

0.11 Two Types of Loop

In Python, we will look at two main ways of repeating code.

while loop

Repeat while a condition is True.

```
while condition:  
    # repeat this
```

This session: **while** | Next session: **for**

for loop

Repeat for a sequence of values.

```
for value in sequence:  
    # repeat this
```



0.12 Our First while Loop

0.12 Our First while Loop

```
count = 1
```

```
while count <= 5:  
    print(count)  
    count = count + 1
```

What do you think this program will display?

1
2
3
4
5



0.13 We Already Know Most of This

0.13 We Already Know Most of This

```
count = 1
```

```
while count <= 5:  
    print(count)  
    count = count + 1
```

count = 1 → **variable**

count <= 5 → **Boolean condition**

count = count + 1 → **update a variable**

The new idea is that Python keeps checking the condition and repeating the indented code while it remains True.



0.14 Trace the Loop

0.14 Trace the Loop

```
count = 1
```

```
while count <= 5:  
    print(count)  
    count = count + 1
```

count	count <= 5	output
1	True	1
2	True	2
3	True	3
4	True	4
5	True	5
6	False	Stop



0.15 What Could Go Wrong?

0.15 What Could Go Wrong?

Look carefully at this:

```
count = 1  
  
while count <= 5:  
    print(count)
```

count starts at 1 and never changes.

The condition therefore remains True.

The program keeps printing 1 forever.

This is an **infinite loop**.



0.15 What Could Go Wrong?

Always ask: **What will eventually make my condition False?**



0.16 Bringing Our Skills Together

0.16 Bringing Our Skills Together

```
def check_age(age):  
    if age >= 18:  
        return "Adult"  
    else:  
        return "Under 18"  
  
again = "yes"  
  
while again == "yes":  
    age = int(input("Enter age: "))  
    print(check_age(age))  
    again = input("Go again? yes/no: ")
```

Can you spot the variables, selection, function, parameter, return value and iteration?



0.17 Our Programming Toolkit

0.17 Our Programming Toolkit

Concept	What does it let us do?
Variables	Remember information
Selection	Make decisions
Functions	Organise and reuse code
Iteration	Repeat code

The real power comes from combining them.



0.18 Today's Focus – while

0.18 Today's Focus – while

Today we are going to learn how to:

- repeat code while a condition is True
- use a counter
- update variables inside a loop
- repeat user input
- combine selection and iteration
- avoid infinite loops
- trace a loop by hand

By the end, you should be able to look at a while loop and explain exactly why it repeats and why it stops.